TECHADEMY

AI IN THE

SDLC

Rethinking AI coding tools and AI agents

Why speeding up the keyboard didn't speed up delivery — and what to redesign instead.

Session deck • adapted from the talk “AI in the SDLC: Rethinking AI Coding Tools & AI Agents”

THE UNCOMFORTABLE FINDING

Developers felt faster. The clock disagreed.

+20%

Perceived speed-up

What the developers believed the AI tooling gave them

−20%

Measured slow-down

What the controlled study actually recorded

A ~40-point gap between felt productivity and real productivity.

Finding from a controlled study on experienced open-source developers, run by a model evaluation and threat research organisation.

Self-reported velocity is not a metric. It is a feeling — and it points the wrong way.

REFRAMING

We have been asking the wrong question

✕

“Can AI write code?”

Settled. Yes, it can — and it does it well.

This question stopped being interesting a while ago.

→

“Why isn't it showing up in delivery?”

The capability is real. The throughput isn't. That gap is a systems problem, not a model problem.

To answer it, you have to look at the whole lifecycle — not the editor.

THE MAP

How software has been delivered for decades

01
Requirements
What are we actually building?

02
Design
What should the system look like?

03
Build
Write the code and the features

04
Test
Verify the behaviour

05
Release
Ship stably and predictably

06
Operate
Run, maintain, keep it alive

This model works. It has produced high-quality software for a very long time.

But notice what it measures: six phases of decision-making, coordination and verification — of which exactly one is typing code.

THE OPEN SECRET

Most of the lifecycle isn't coding. It's waiting.

⏸ Dev → Product

A developer sits on a story that isn't clear yet, waiting for the product team to define what "done" means.

⏸ Ops → Dev

Operations waits on the development team for a release that is ready to actually go out.

⏸ QA → Build

QA waits for a build to appear before it can test anything at all.

And it all happens across fragmented tools and platforms.

What QA sees isn't what production sees. What staging runs isn't what the product team is poking at. Every handoff is a queue, and every environment is a different truth.

AMDAHL'S LAW

Make one box faster and the rest absorbs the gain

> ***Amdahl's Law*** *is a principle in computer science used to find the maximum theoretical speedup of a system when only a portion of it is improved. It states that the overall performance gain is ultimately limited by the* ***sequential (un-parallelizable) part*** *of the task.*

TODAY



AI ON THE BUILD PHASE ONLY



Coding three times faster is real. It is also worth almost nothing while the surrounding process stays the same.

TECHADEMY

TWO WAYS TEAMS GET THIS WRONG

The delegation spectrum

Over-delegation

Hand the model one huge, ambiguous problem and hope.

The useful zone

Under-delegation

Keep every judgement call human; rent AI by the function.

Both ends feel like they should work. Both end up slower than the team expected.

TECHADEMY

FAILURE MODE 01

Over-delegation: the prompt full of silent decisions

> *“Build me an e-commerce platform.”*

Every decision this prompt does not make, the model makes for you — silently:

- Payments
- Authentication
- Shipping
- Data model
- Failure behaviour

Net effect on the lifecycle: slower.

1. **Requirements and design get skipped**
   Work that belongs in the first two phases is quietly delegated to a model that was never told the constraints.

2. **Thousands of lines nobody has read**
   Volume of output stops correlating with progress the moment review capacity is exceeded.

3. **Review becomes the new bottleneck**
   The team spends its time in back-and-forth over decisions a model made, and the build-phase gains evaporate.

FAILURE MODE 02

Under-delegation: good code, unchanged economics

What the senior developer keeps

- All of the planning
- All of the task decomposition
- All of the architectural judgement

What the AI is allowed to touch

- “Write this function.”
- “Review this code for SQL injection.”
- Narrow, well-scoped, low-context tasks

The code is good. But the intellectual heavy lifting is still 100% human — and it sits in requirements and design, the phases you never touched. You bought a faster keyboard for the smallest part of the job.

THE SHIFT

Don't bolt AI onto the lifecycle.

Redesign the lifecycle around it.

The productivity gain never came from a better model or a better tool. It comes from removing the friction between phases — and from moving the human from typing to validating.

TECHADEMY

REDESIGN 01 — REQUIREMENTS & DESIGN

Stop generating lines. Start generating understanding.

Synthesise the unstructured

Surveys · user reports · support email · stakeholder conversations

- Surface real user behaviour and usage patterns
- Identify where users actually get stuck
- Draft user stories that feed the next set of features

Mine production reality

Logs · bug reports · incident history

- Agents cluster failures and identify root causes
- You learn what genuinely works and what fails in production
- That evidence flows back into the next requirement

Operate → Requirements the loop most teams never close

REDESIGN 02 — BUILD

Vibe coding doesn't scale. Specs do.

Instead of asking AI to build entire systems, aim it at small, well-defined tasks — and make the definition machine-readable.

01
Intent
What the business actually wants

▶

02
Specification
Written so a model can read and follow it

▶

03
Tasks
Small, bounded, independently reviewable

▶

04
Verified change
Reviewed against the spec, not against vibes

Spec-driven development isn't just task breakdown. It's turning intent into an artefact the model — and the reviewer — can both hold to.

TECHADEMY

REDESIGN 02 — BUILD

The agent is not the system. The harness is.

SPECIFICATION → agent + harness (the tools, context and guardrails around the model)

Research subagent

Investigates the topic and the dependencies you're pulling in before anything is written.

MCP subagent

Talks to MCP servers to pull the data your team actually needs from the systems it lives in.

Code-editing subagent

Makes the change — the new features and functions the spec asked for.

AGENTS.md
Shared context, carried across teams instead of living in one engineer's head.

Skills
The same output every time — whether the model is local, private or in the cloud.

TECHADEMY

REDESIGN 03 — TEST

Manual testing is the classic bottleneck

1. **Test data on demand**

   If a model can generate code, it can generate the awkward, edge-case data your unit tests never had.

2. **Cases straight from the story**

   Derive test cases directly from the user story — the same artefact that drove the build.

3. **Triage at 3 a.m.**

   Applications emit more log data than anyone can read. A stack trace is exactly the kind of thing a model is good at explaining when the system goes down.

QA stops being the queue everyone waits behind, and becomes the team that decides what "correct" means.

REDESIGN 04 — RELEASE & OPERATE

Infrastructure as code is already well within reach

Ansible

Playbooks to patch and update virtual machines.

Kubernetes YAML

Manifests to deploy containerised applications into hybrid cloud.

Why this phase is easy pickings

Models are unusually well trained on infrastructure as code — it is public, highly patterned, and heavily documented. **Everything described here is possible with today's agents, not a future generation of them.**

THE SLEEPER USE CASE

Legacy systems nobody left understands

The situation

- Business-critical code, decades old
- Original developers long gone
- Written in a language your team may not read
- No documentation worth the name
- Nobody dares touch it

What AI actually does here

- Explains what the code does, function by function
- Reverse-engineers the system's intent and behaviour
- Bridges a language you don't know into one you do
- Gives you a defensible path forward — not a rewrite gamble

Keep the lifecycle. Use AI to recover the understanding the lifecycle assumed you had.

WHAT ACTUALLY CHANGES

The role moves. So should the scoreboard.

The human moves from ~~typing~~ to **validating, and coordinating across teams.**

STOP MEASURING

~~**Lines of code generated**~~

Output volume is not outcome. It never was.

START MEASURING

- **System health in production**
- **Code maintainability and complexity**
- **Time to ship a change or a new feature**

All of it in service of one thing: making life easier for the developer — and for every team the developer depends on.

TECHADEMY

TAKE THIS AWAY

Three things worth arguing about

01

Perceived speed is not speed

If your AI adoption case rests on how fast the team feels, you have no evidence. Instrument the life cycle instead.

02

The bottleneck is between the boxes

Waiting, handoffs, fragmented tools and mismatched environments — not typing. Optimise the queue, not the keyboard.

03

Specs and harnesses beat bigger prompts

Small bounded tasks, a readable specification, sub agents with real tools, shared context. Then measure outcomes, not output.

The gain was never in the model. It was in the life cycle you were willing to redesign.

TECHADEMY